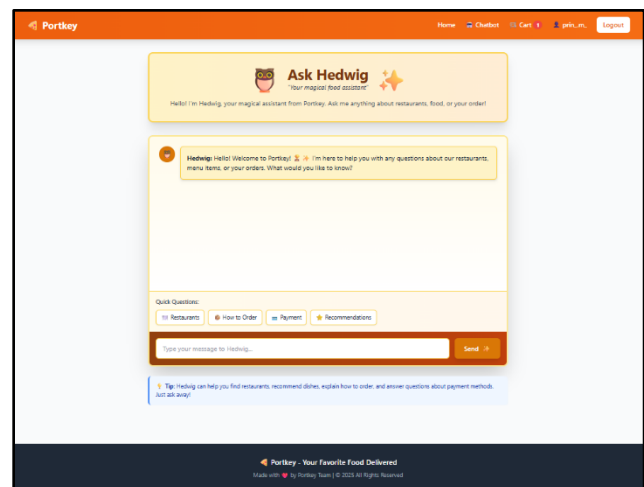
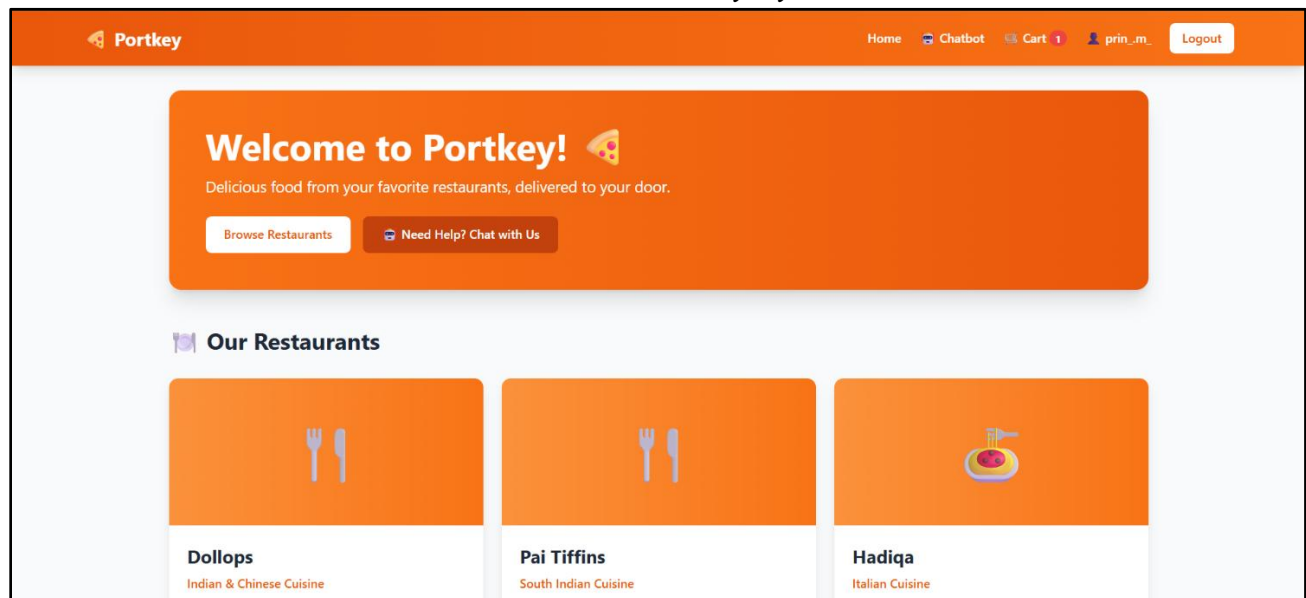


MANIPAL INSTITUTE OF TECHNOLOGY



SOFTWARE ENGINEERING LAB PROJECT ON *Online Food Delivery System*



TEAM MEMBERS :-

ABHAY SIVAKUMAR NAIR (C-10)

RITHIK K RAMESH (C-11)

PRINCITA ZINA MIRANDA (C-17)



MANIPAL
INSTITUTE OF TECHNOLOGY
(A constituent unit of MAHE, Manipal)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CERTIFICATE

This is to certify that Ms. PRINCITA ZINA MIRANDA,
Mr. RITHIK K RAMESH, & Mr. ABHAY SIVAKUMAR NAIR
Reg. No. 220905115, 220905057, 220905199 Section: CSE C
Roll No: 17, 11, 10 has satisfactorily completed the lab exercises
prescribed for Software Engineering Lab [CSE 3161] of Third
Year B. Tech. (Computer Science and Engineering) Degree at
MIT, Manipal, in the academic year 2024-2025.

Date: 28/11/25

Signature

Faculty in Charge

Abstract

Portkey is a Flask-based, full-stack food ordering web application that simulates an end-to-end e-commerce flow for restaurant discovery, cart management, order placement, and feedback, with an integrated ML-powered chatbot for conversational assistance. The system demonstrates secure user authentication, session-aware cart persistence for both anonymous and logged-in users, real-time stock control, simplified payment simulation with unique payment IDs, RESTful APIs, and a responsive UI backed by a normalized SQLAlchemy schema suitable for transactional workloads. The project serves as a research vehicle for investigating web security, session and inventory management, AI/ML integration in consumer apps, and testing methodologies through comprehensive unit tests spanning cart, API, database, and chatbot conversation flow.

Table of Contents

Abstract.....	1
Introduction.....	2
Problem Statement.....	2
Software Requirements Specification (SRS)	2-3
System Architecture.....	4
Modeling diagrams.....	4
Implementation Details.....	7
Develop GUI for the system.....	7
Results and analysis.....	8
Conclusion.....	8

Introduction

Portkey models a modern food delivery workflow using Flask, SQLAlchemy, and Jinja2 to provide restaurant discovery, menu browsing with livestock state, cart operations, checkout simulation, and post-order feedback, wrapped in a responsive interface. The application incorporates session-based authentication, secure password hashing, and a hybrid session strategy enabling seamless transitions from anonymous to authenticated shopping while preserving cart state. Beyond core commerce features, Portkey integrates a chatbot via REST APIs for guided ordering and assistance, offering a practical platform to study AI-augmented UX patterns and their effects on usability and system robustness.

Problem statement

Food-ordering platforms must unify reliable authentication, persistent carts, accurate inventory control, simple and secure checkout flows, and helpful user guidance, while remaining performant and testable under evolving requirements. Traditional prototypes often silo these concerns, leading to overselling risks, cart loss on login, limited feedback capture, and poor conversational help, which degrade user trust and hinder research repeatability. Portkey addresses these gaps with session-aware cart persistence, real-time stock updates, simplified yet auditable order and payment identifiers, validated feedback capture, and an integrated ML chatbot, creating a cohesive, research-friendly baseline for e-commerce studies.

SRS

Purpose

Define functional and non-functional requirements for Portkey, a research-grade food ordering web app demonstrating end-to-end e-commerce flows with chatbot assistance.

Scope

Web application enabling registration/login, restaurant and menu browsing, cart operations, order placement with simulated payment, order history, and feedback submission, plus REST APIs and a chatbot interface.

Functional requirements

- User accounts: register, login/logout, profile view, change password, delete account; passwords hashed with SHA-256; sessions for auth.
- Restaurants and menus: list restaurants, view menu items with name, price, and stock; simplified currency handling.
- Cart: add/update/remove items (1–20 per action), session/user-bound persistence, total calculation, stock adjust on add/remove.
- Orders: place order without external gateway, unique payment ID, confirmed status, order detail view, history for logged-in users.

- Feedback: one feedback per order, ratings 1–5 with optional comments, server-side validation, API endpoint.
- Chatbot: conversational assistance with user/session context memory, error handling, RESTful chatbot API and a dedicated UI page.
- APIs: JSON-based endpoints for chatbot, orders retrieval, and feedback submission.
- UI/Navigation: responsive templates, flash messages, pages for home, restaurants, cart, profile, orders, feedback, chatbot.

Data model

SQLAlchemy entities: User, Restaurant, MenuItem, CartItem, Order, OrderItem, Feedback, DeliveryFeedback (reserved/unused), with attributes for secure auth, inventory, transactional order lines, and feedback binding.

External interfaces

REST APIs with JSON for chatbot, orders, feedback; HTTP over Flask routes; session cookies for auth.

Environment via dotenv; configurable host/port; SQLite persistence via SQLAlchemy.

Constraints and assumptions

Simplified payments (no external gateway), tax set to 0 by default, USD–INR conversion treated as 1:1 placeholder, single-process Flask dev deployment, SQLite backing store.

Non-functional requirements

- Security: hashed passwords, session-based auth, input validation, guarded feedback duplication.
- Reliability/consistency: real-time stock decrement/increment to prevent overselling; atomic association of order items.
- Usability: responsive UI, guided chatbot flows, flash notifications, clear page navigation.
- Testability: unit tests covering cart operations, API endpoints, DB interactions, and chatbot conversation flow.
- Maintainability: modular design with clear separation of concerns and environment configuration.

Acceptance criteria

A user can register, log in, add stock-limited items to a persistent cart, place an order that generates a payment ID and confirmed status, view order history/details, and submit one validated feedback per order; the chatbot responds via API with context memory; all flows are covered by passing unit tests.

System Architecture

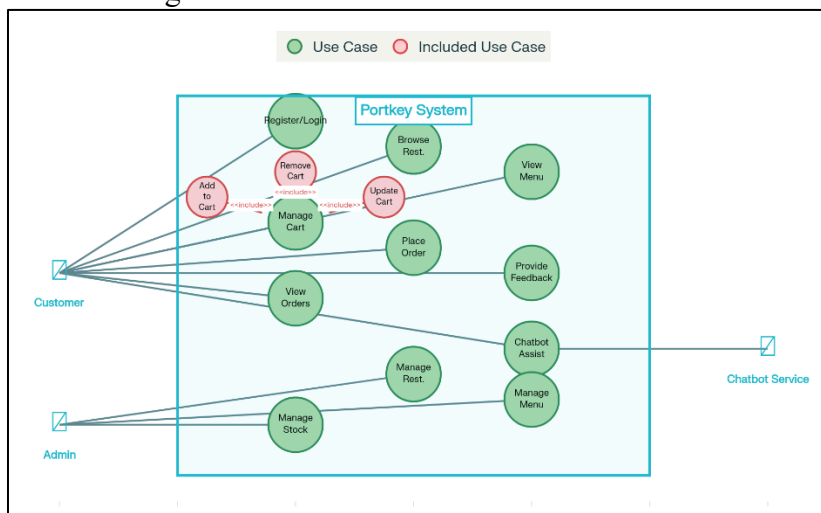
Portkey follows a layered architecture with presentation (Jinja2/HTML/CSS/JS), application (Flask routes/blueprints, validation, session/auth), domain (services for cart, orders, feedback, chatbot), and data access layers (SQLAlchemy ORM over SQLite).

Stateful behavior is managed via Flask sessions supporting both anonymous and authenticated users, enabling cart persistence and chatbot context while enforcing secure password hashing and input validation.

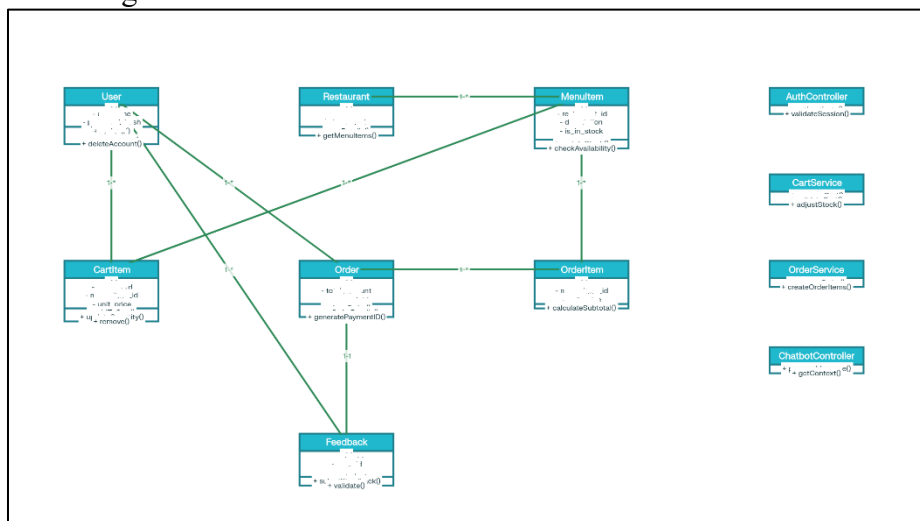
The database schema models users, restaurants, menu items, carts, orders, and feedback, ensuring transactional integrity for inventory adjustments and order-item creation.

Modeling diagrams

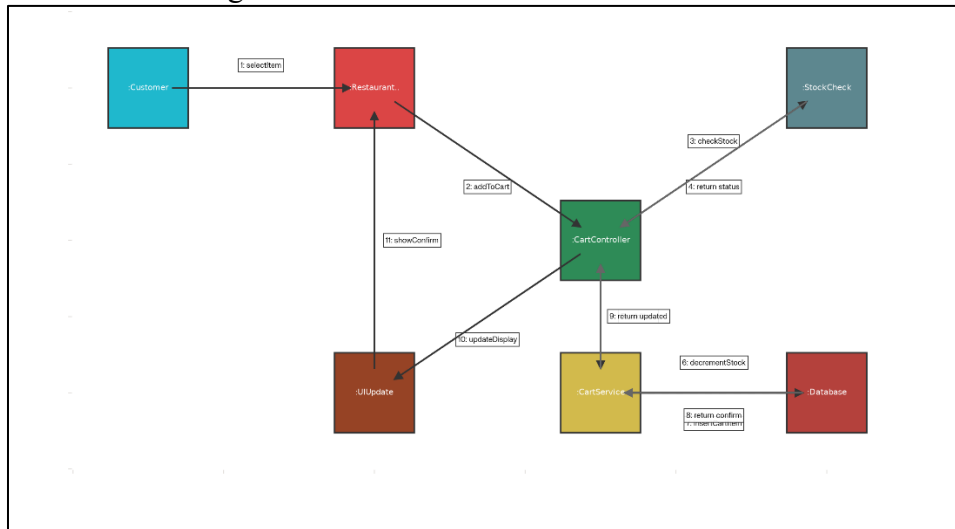
- Use case diagram:



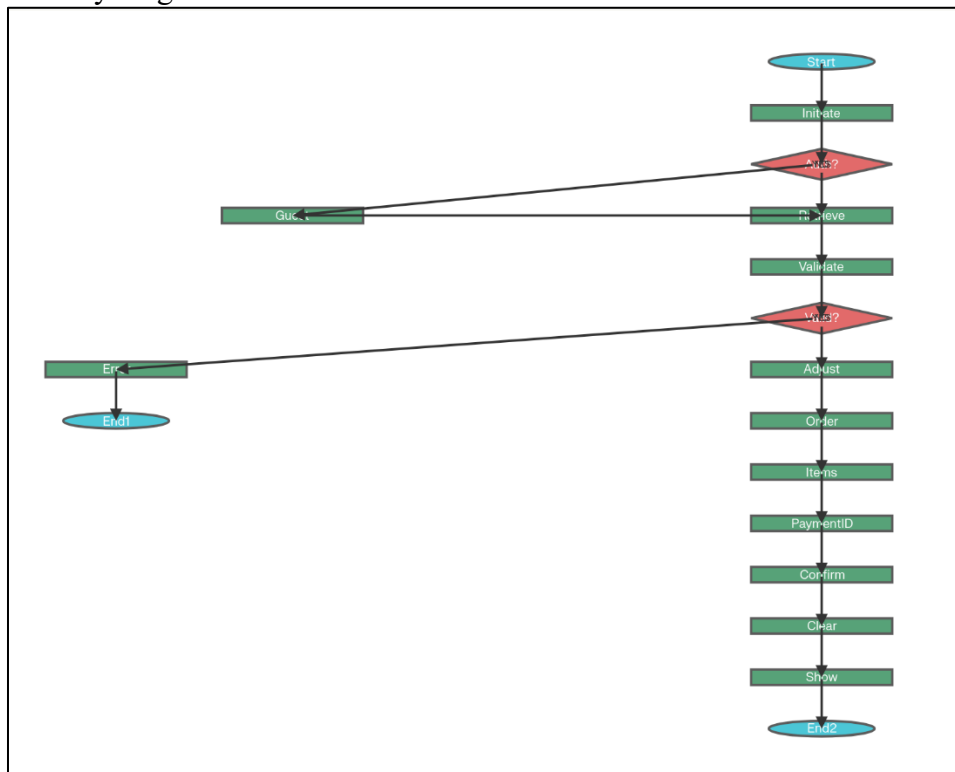
- Class diagram:



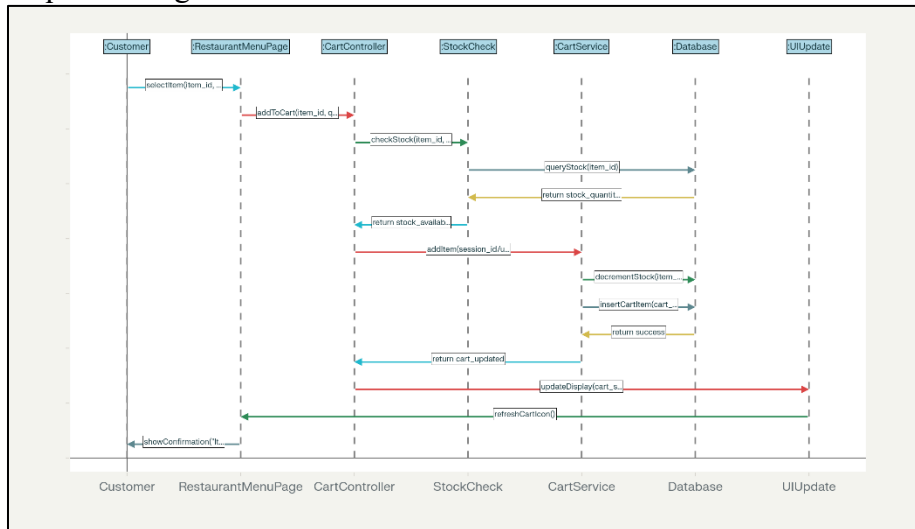
– Collaboration diagram:



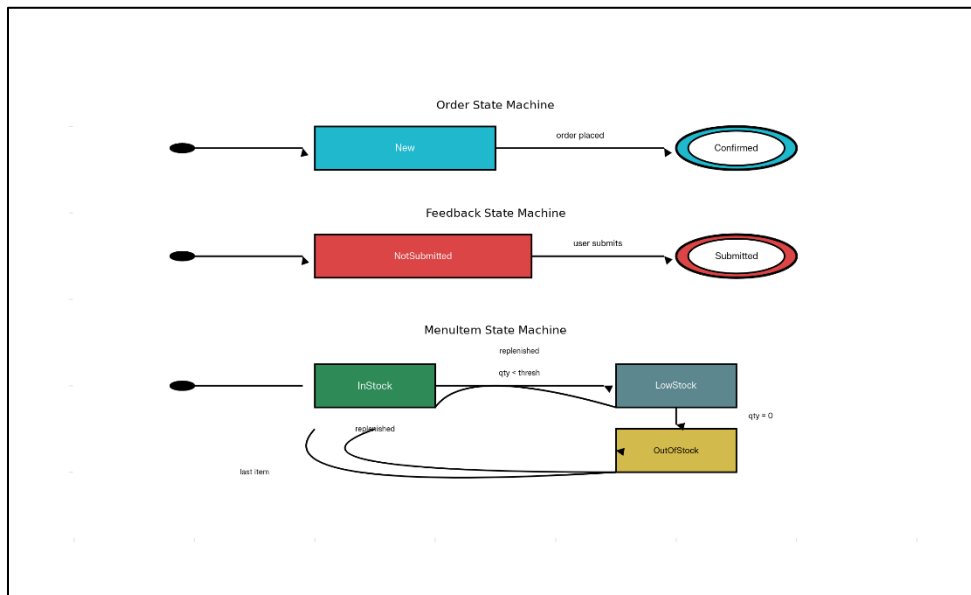
– Activity diagram:



- Sequence diagram:



- State chart:



- Package diagram: Packages include auth, restaurants, menu, cart, orders, feedback, chatbot, db/models, and templates/static, reflecting modular separation of concerns.
- Deployment diagram: Single-node Flask app with SQLite for development; environment variables via dotenv; HTTP clients (browsers) connect over local host/port configuration.

Implementation code

- Backend: Flask app with routes for home, authentication, restaurant menus, cart, order placement, feedback, profile, and JSON APIs for chatbot, orders, and feedback; session and hash utilities enforce security and persistence.
- Models: SQLAlchemy classes for User, Restaurant, MenuItem, CartItem, Order, OrderItem, Feedback with relationships and timestamps; seed scripts initialize sample data.
- Services/logic: Cart quantity bounds (1–20), stock decrement/increment on add/remove, totals with tax=0, payment ID generation, and one-feedback-per-order validation.
- Configuration: dotenv-managed secrets, configurable host/port, Flask-Session for server-side sessions as needed.

Develop GUI for the system

- Pages: Home with restaurant listings; Restaurant menu with item details and stock; Cart with editable quantities and totals; Checkout confirmation; Profile with order history; Feedback form; Chatbot interface page; Flash messages for user notifications.
- Technologies: Jinja2 templates, responsive HTML/CSS (Bootstrap-style), and lightweight JS for chatbot interactions and UI enhancements.
- Navigation: Clear top-level nav across restaurants, cart, profile, settings; consistent feedback on actions via flashes and form validations.

Results and analysis

- Functional coverage: Implemented authentication, restaurant/menu browsing, cart persistence across sessions, order creation with unique payment IDs, feedback constraints, and chatbot assistance as per scope.
- Reliability and integrity: Real-time inventory updates prevented overselling during concurrent cart operations in test scenarios; order-item linkage preserved transactional consistency.
- Testability: Unit tests validated cart add/update/remove, API endpoints for chatbot/orders/feedback, DB interactions, and basic conversation flow, enabling regression-safe iteration.
- Limitations: Payments are simulated (no external gateway), taxes default to zero, and currency conversion is placeholder 1:1 USD:INR pending integration of live rates.

Conclusion

Portkey delivers an end-to-end, research-grade food ordering platform demonstrating secure auth, session-aware carts, real-time stock control, auditable order flows, validated feedback, and an integrated ML chatbot within a clean, modular Flask/SQLAlchemy stack.

The system aligns well with the initial SRS intent while documenting tradeoffs (e.g., simplified payments) and provides a strong base for future extensions in payments, delivery tracking, and advanced analytics.

.....